

Python for Data Science

Buổi 2: IPython/Jupyter Nâng Cao & NumPy Foundation

PGS-TS Nguyễn Mạnh Hùng

August 19, 2026

Cấu trúc Buổi 2

- 1 Ôn tập Buổi 1 và định vị NumPy trong Data Science Workflow
- 2 Module 4: IPython cho Data Science
- 3 Module 5: NumPy Fundamentals – ndarray và Array Creation
- 4 Module 6: NumPy Computation – UFuncs, Aggregation, Broadcasting
- 5 Module 7: Boolean Masking, Fancy Indexing, Sorting, Structured Arrays
- 6 Module 8: Ứng dụng NumPy – Hồi quy tuyến tính & SVD

Lưu ý phạm vi

Buổi học tập trung nền tảng tính toán số với NumPy – công cụ đứng sau Pandas, Scikit-Learn và tensor của PyTorch, khép lại bằng ví dụ kinh điển: huấn luyện hồi quy tuyến tính bằng Gradient Descent, Normal Equation và SVD. Buổi học chưa đi sâu vào GPU computing hay tensor tự động lấy đạo hàm (autograd).

Nhắc lại Buổi 1

- Data Science Workflow: Business → Data Collection → Cleaning → EDA → Modeling → Deployment.
- Handbook Overview: IPython → NumPy → Pandas → Matplotlib → Scikit-Learn.
- Python OOP: mọi thứ trong Python là object – kể cả `numpy.ndarray`.

Câu hỏi mở đầu

Trước khi làm sạch hay trực quan hóa dữ liệu, làm thế nào để *tính toán* trên hàng triệu con số một cách nhanh chóng?

Vì sao không dùng list Python thuần?

```
import time

n = 2_000_000
data = list(range(n))

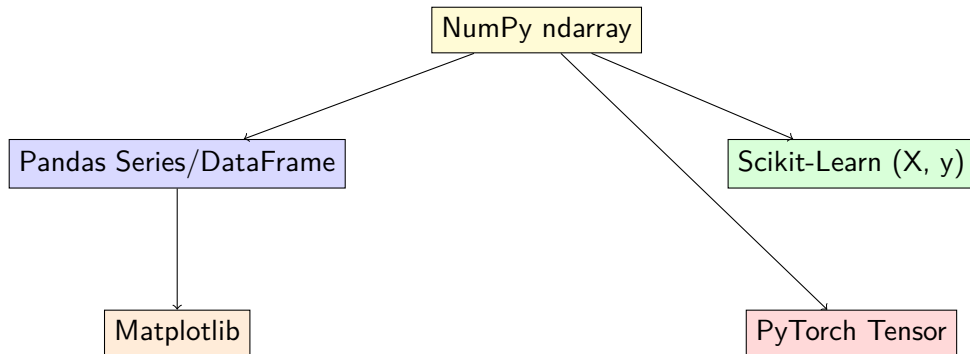
start = time.time()
squared = [x ** 2 for x in data]          # vòng lặp Python thuần
print("List comprehension:", time.time() - start)

import numpy as np
arr = np.arange(n)

start = time.time()
squared_np = arr ** 2                    # vectorized
print("NumPy vectorized:", time.time() - start)
```

NumPy thường nhanh hơn hàng chục lần vì dữ liệu được lưu liên tục trong bộ nhớ (contiguous memory) với kiểu dữ liệu cố định (fixed type), và phép tính được thực hiện ở tầng C thay vì vòng lặp interpreter Python.

NumPy trong hệ sinh thái Data Science



NumPy là nền tảng tính toán số của gần như toàn bộ hệ sinh thái Data Science/Deep Learning trong Python (Harris et al., 2020).

Module 4: IPython cho Data Science

Chuẩn đầu ra (Learning Outcomes)

Sau khi hoàn thành module này, học viên có khả năng:

- ➊ Sử dụng help/documentation và tab completion để khám phá API nhanh.
- ➋ Vận dụng magic commands để đo hiệu năng và gỡ lỗi (debug) code.
- ➌ Sử dụng lịch sử input/output và shell command ngay trong notebook.

Help và Documentation

```
import numpy as np
```

```
np.array?           # xem docstring của ham/object  
np.array??          # xem cả source code (nếu có)  
np.ar<TAB>           # tab completion: gợi ý các hàm bắt đầu bằng "ar"
```

- ? và ?? là tính năng đặc trưng của IPython, không có trong Python shell chuẩn (Pérez & Granger, 2007).
- Tab completion giúp khám phá API của thư viện lớn (NumPy, Pandas) mà không cần rời notebook để tra tài liệu.

Magic Commands: đo hiệu năng

```
%timeit [x**2 for x in range(100000)]    # lap nhieu lan, do thoi gian trung binh
%time  squared = arr ** 2                 # do 1 lan duy nhat

%%time
total = 0
for i in range(1_000_000):
    total += i
```

- `%timeit`: chạy lệnh nhiều lần, phù hợp so sánh hiệu năng giữa các cách viết code.
- `%time`: đo một lần, phù hợp cho thao tác tốn thời gian (đọc file lớn, train model).
- `%%time` (cell magic): áp dụng cho toàn bộ cell thay vì một dòng.

Magic Commands: làm việc và gỡ lỗi

```
%matplotlib inline      # hiện biểu đồ ngay trong notebook
%run script.py           # chạy một file .py như một script
%load script.py          # nạp nội dung file vào cell hiện tại
%pwd                     # thu mục hiện tại
%who                      # liệt kê biến đang có trong session

%xmode Verbose            # hiện traceback chi tiết hơn khi có lỗi
%debug                    # mở interactive debugger ngay tại vị trí lỗi
```

%debug mở trình gỡ lỗi tương tác (dựa trên pdb) ngay tại dòng gây lỗi – hữu ích khi debug pipeline xử lý dữ liệu hoặc vòng lặp huấn luyện dài.

Input/Output History và Shell Commands

```
2 + 3
Out[1]      # 5

_           # ket qua cua lenh truoc do (Out gan nhat)
--          # ket qua cua lenh truoc do nua
Out[1]      # tham chieu truc tiep theo so thu tu

!pwd                # chay lenh shell truc tiep
!ls *.csv
files = !ls *.csv    # gan ket qua shell command vao bien Python
```

IPython lưu lại toàn bộ lịch sử input (In) và output (Out), đồng thời cho phép gọi lệnh shell (!) và trộn lẫn với biến Python – rất tiện khi thao tác nhanh với file dữ liệu.

Tổng kết Module 4

- `?`, `??`, Tab completion: khám phá API nhanh.
- `%timeit`, `%time`, `%%time`: đo hiệu năng.
- `%xmode`, `%debug`: gỡ lỗi tương tác.
- `_`, `Out[n]`, `!command`: lịch sử và shell.

Thông điệp

Thành thạo IPython giúp vòng lặp thử nghiệm (experiment loop) trong Data Science nhanh hơn đáng kể so với chạy script truyền thống.

Module 5: NumPy Fundamentals

Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- 1 Giải thích vì sao ndarray lưu trữ và tính toán hiệu quả hơn list.
- 2 Tạo array bằng nhiều phương pháp khác nhau (array, zeros, arange, random...).
- 3 Truy cập, cắt (slice) và định hình lại (reshape) array nhiều chiều.
- 4 Phân biệt view và copy khi thao tác trên array.

ndarray là gì?

Khái niệm

`ndarray` (N-dimensional array) là cấu trúc dữ liệu lõi của NumPy: một mảng đồng nhất kiểu dữ liệu (fixed type), được lưu liên tục trong bộ nhớ.

	Python list	NumPy ndarray
Kiểu phần tử	Có thể khác nhau	Cố định (dtype)
Bộ nhớ	Rải rác (con trỏ)	Liên tục (contiguous)
Phép toán số học	Cần vòng lặp	Vectorized

Tạo array từ dữ liệu có sẵn

```
import numpy as np

a = np.array([1, 2, 3, 4])
print(a.dtype)                # int64

b = np.array([1, 2, 3.5])
print(b.dtype)                # float64 (upcasting)

c = np.array([1, 2, 3], dtype="float32")
matrix = np.array([[1, 2, 3], [4, 5, 6]]) # array 2 chieu
```

NumPy tự động chọn kiểu dữ liệu chung (upcasting) khi các phần tử có kiểu khác nhau; có thể ép kiểu tường minh bằng tham số dtype.

Tạo array từ công thức dựng sẵn

```
np.zeros((3, 4))           # array 3x4 toàn số 0
np.ones((2, 3), dtype=int) # array 2x3 toàn số 1
np.full((2, 2), 7)         # array 2x2 toàn giá trị 7
np.eye(3)                  # ma trận đơn vị 3x3

np.arange(0, 20, 2)        # [0, 2, 4, ..., 18]
np.linspace(0, 1, 5)       # 5 giá trị chia đều trong [0, 1]

np.random.seed(42)
np.random.random((2, 2))   # giá trị ngẫu nhiên trong [0, 1)
np.random.normal(0, 1, (2, 2)) # phân phối chuẩn
np.random.randint(0, 10, (3, 3)) # số nguyên ngẫu nhiên
```

Thuộc tính của Array

```
x = np.random.randint(10, size=(3, 4, 5))

print(x.ndim)      # 3 - so chieu
print(x.shape)     # (3, 4, 5) - kích thước từng chiều
print(x.size)      # 60 - tổng số phần tử
print(x.dtype)     # int64 - kiểu dữ liệu
print(x.itemsize)  # số byte mỗi phần tử
print(x.nbytes)    # tổng số byte (itemsize * size)
```

So sánh với `tensor.shape`, `tensor.dtype` trong PyTorch – cùng một tư duy thiết kế thuộc tính (attribute) đã học ở Module 3 Buổi 1.

Array Indexing: truy cập từng phần tử

```
x1 = np.array([5, 0, 3, 3, 7, 9])
print(x1[0])          # 5
print(x1[-1])         # 9 - phần tử cuối

x2 = np.array([[3, 5, 2], [7, 6, 8]])
print(x2[0, 0])       # 3
print(x2[1, -1])      # 8

x2[0, 0] = 99         # gán giá trị mới
```

Chỉ số nhiều chiều dùng cú pháp `array[hàng, cột]`, khác với `list[i][j]` lồng nhau trong Python thuần.

Array Slicing: subarray

```
x = np.arange(10)
print(x[:5])          # [0 1 2 3 4]
print(x[5:])          # [5 6 7 8 9]
print(x[::2])         # [0 2 4 6 8] - buoc nhay 2
print(x[::-1])        # dao nguoc mang
```

```
x2 = np.arange(12).reshape(3, 4)
print(x2[:2, :2])     # 2 hang dau, 2 cot dau
print(x2[:, 0])       # toan bo cot dau tien
```

View vs Copy – điểm khác biệt quan trọng

```
x = np.arange(10)
sub = x[2:5]          # slicing trả về VIEW, không phải copy
sub[0] = 99
print(x)              # [0 1 99 3 4 5 6 7 8 9] - x cũng bị thay đổi!

sub_copy = x[2:5].copy() # tạo bản sao thực sự
sub_copy[0] = -1
print(x)              # x không đổi
```

Lỗi thường gặp

Khác với Python list, slicing trên ndarray trả về *view* chia sẻ cùng vùng nhớ với array gốc. Sửa dữ liệu trong subarray sẽ làm thay đổi array gốc – cần dùng `.copy()` khi muốn dữ liệu độc lập.

Reshaping Arrays

```
grid = np.arange(1, 10).reshape(3, 3)
print(grid)

x = np.array([1, 2, 3])
print(x.reshape(1, 3))      # row vector: [[1 2 3]]
print(x.reshape(3, 1))      # column vector

flat = grid.reshape(-1)      # -1: NumPy tự tính kích thước còn lại
```

Tham số `-1` trong `reshape` cho phép NumPy tự suy ra kích thước còn lại, tiện dụng khi chuẩn bị dữ liệu đầu vào cho mô hình (ví dụ: làm phẳng ảnh trước khi đưa vào fully-connected layer).

Concatenation và Splitting

```
x = np.array([1, 2, 3])
y = np.array([4, 5, 6])
print(np.concatenate([x, y]))           # [1 2 3 4 5 6]

grid = np.array([[1, 2], [3, 4]])
print(np.vstack([grid, [5, 6]]))        # ghep theo hang
print(np.hstack([grid, [[9], [9]]]))    # ghep theo cot

x = np.arange(8)
a, b, c = np.split(x, [3, 5])           # cat tai vi tri 3 va 5
```

Tổng kết Module 5

- ndarray: fixed type, contiguous memory → tính toán nhanh.
- Tạo array: array, zeros/ones/full/eye, arange/linspace, random.*.
- Thuộc tính: ndim, shape, size, dtype.
- Indexing/Slicing nhiều chiều; **slicing trả về view, không phải copy**.
- reshape, concatenate/vstack/hstack, split.

Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- 1 Giải thích và sử dụng ufunc để thay thế vòng lặp Python.
- 2 Tính toán các phép aggregation (sum, mean, std, min/max) trên nhiều chiều.
- 3 Áp dụng broadcasting để tính toán trên array có shape khác nhau.

Vấn đề: vòng lặp Python rất chậm

```
def compute_reciprocals(values):
    output = np.empty(len(values))
    for i in range(len(values)):
        output[i] = 1.0 / values[i]
    return output

big_array = np.random.randint(1, 100, size=1_000_000)
%timeit compute_reciprocals(big_array)      # chậm: vài giây

%timeit (1.0 / big_array)                  # nhanh: vài chục milli-giây
```

Nguyên nhân: mỗi vòng lặp Python phải kiểm tra kiểu dữ liệu và gọi hàm động (dynamic typing) cho từng phần tử – rất tốn kém khi lặp lại hàng triệu lần.

Vector hóa (Vectorization) là gì?

Định nghĩa

Vector hóa là áp dụng *một phép toán duy nhất* lên *toàn bộ* mảng dữ liệu cùng lúc (ví dụ 1.0 / big_array), thay vì viết vòng lặp for xử lý từng phần tử như compute_reciprocals().

- Vòng lặp *vẫn xảy ra* về bản chất, nhưng được **đẩy xuống tầng C** đã biên dịch sẵn, thay vì chạy bằng interpreter Python (kiểm tra kiểu & gọi hàm động cho từng phần tử, như trang trước).
- Không phải phép màu "song song hóa phần cứng" – lợi ích chính là loại bỏ chi phí lặp của Python interpreter hàng triệu lần.

Khái niệm xuyên suốt môn học

Không chỉ ufunc: broadcasting, boolean masking (trang sau), .str accessor xử lý chuỗi (Buổi 7), và hầu hết phép toán trên Series/DataFrame Pandas đều dựa trên cùng nguyên lý này.

Universal Functions (ufuncs)

```
x = np.arange(4)
print(x + 5)      # array([5, 6, 7, 8])
print(x * 2)      # array([0, 2, 4, 6])
print(x ** 2)     # array([0, 1, 4, 9])
print(-x)         # array([ 0, -1, -2, -3])

print(np.exp(x))  # e^x tung phan tu
print(np.log(x + 1)) # log tung phan tu
print(np.sin(x))  # sin tung phan tu
```

Khái niệm

Ufunc (universal function) áp dụng một phép toán lên *từng phần tử* của array theo cách vectorized, được cài đặt tối ưu ở tầng C (Harris et al., 2020).

Aggregation: tổng hợp thống kê

```
x = np.random.random(100)
print(np.sum(x))           # tổng
print(np.min(x), np.max(x)) # giá trị nhỏ nhất / lớn nhất
print(np.mean(x), np.std(x)) # trung bình, độ lệch chuẩn
```

```
grid = np.random.random((3, 4))
print(grid.sum())           # tổng toàn bộ
print(grid.sum(axis=0))     # tổng theo từng cột
print(grid.sum(axis=1))     # tổng theo từng hàng
```

axis=0: tổng hợp *dọc theo* chiều 0 (kết quả gọn theo hàng); axis=1: tổng hợp *dọc theo* chiều 1 (kết quả gọn theo cột) – đây là nguồn nhầm lẫn phổ biến nhất khi mới học NumPy/Pandas.

Ví dụ thực tế: thống kê nhanh trên dữ liệu thực

```
# Ví dụ minh họa: số liệu tên trẻ sơ sinh theo năm (US Baby Names)
# (chi tiết thực hành dạy dự sẽ làm với Pandas ở Buổi 4)
counts = np.array([9217, 9655, 9436, 9166, 8798]) # số lượng theo 5 năm

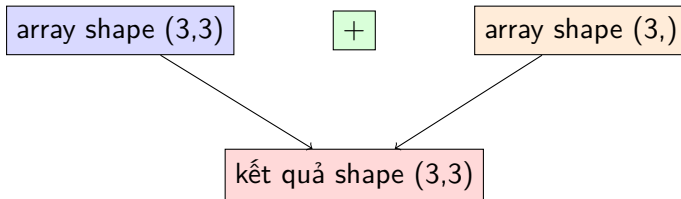
print("Trung bình:", counts.mean())
print("Độ lệch chuẩn:", counts.std())
print("Năm có số lượng cao nhất:", counts.argmax())
```

Dữ liệu minh họa lấy cảm hứng từ case study *US Baby Names* (Python for Data Analysis, Module 4) – sẽ được khai thác đầy đủ với Pandas ở các buổi sau.

Broadcasting là gì?

Khái niệm

Broadcasting cho phép NumPy thực hiện phép toán giữa các array có *shape khác nhau* bằng cách "mở rộng" array nhỏ hơn một cách ngầm định, không tốn thêm bộ nhớ sao chép dữ liệu.



Broadcasting: ví dụ

```
a = np.array([[0, 0, 0], [10, 10, 10], [20, 20, 20]]) # shape (3,3)
b = np.array([1, 2, 3])                               # shape (3,)
```

```
print(a + b)
# [[ 1  2  3]
#  [11 12 13]
#  [21 22 23]]
```

```
scalar_result = a + 5    # scalar cũng được "broadcast" thành (3,3)
```

Quy tắc broadcasting (rút gọn): so sánh shape từ phải sang trái; hai chiều tương thích nếu bằng nhau hoặc một trong hai bằng 1.

Ứng dụng: chuẩn hóa dữ liệu bằng broadcasting

```
X = np.random.random((10, 3))    # 10 mau, 3 feature

mean = X.mean(axis=0)             # shape (3,) - trung bình tung cot
std = X.std(axis=0)               # shape (3,) - do lech chuan tung cot

X_scaled = (X - mean) / std       # broadcasting: (10,3) voi (3,)
```

So sánh với `StandardScaler` viết bằng vòng lặp Python thuần ở Buổi 1 (Module 3): cùng ý tưởng chuẩn hóa dữ liệu, nhưng broadcasting giúp thực hiện vectorized, không cần vòng lặp for tường minh – đây chính là cách `sklearn.preprocessing.StandardScaler` cài đặt bên trong.

Tổng kết Module 6

- Ufunc: thay vòng lặp Python bằng phép toán vectorized, nhanh hơn nhiều bậc.
- Aggregation: `sum/mean/std/min/max`, chú ý `axis=0` và `axis=1`.
- Broadcasting: tính toán giữa array khác shape mà không cần sao chép dữ liệu.

Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- ➊ Dùng phép so sánh và boolean mask để lọc dữ liệu theo điều kiện.
- ➋ Dùng fancy indexing để truy cập nhiều phần tử không liên kề.
- ➌ Sắp xếp array và hiểu vai trò của structured array như cầu nối tới Pandas.

So sánh như một ufunc

```
x = np.array([1, 2, 3, 4, 5])
print(x < 3)          # array([ True,  True, False, False, False])
print(x == 3)         # array([False, False,  True, False, False])

print(np.sum(x < 3))   # 2 - đếm số phần tử thỏa điều kiện
print(np.any(x > 4))   # True
print(np.all(x > 0))   # True
```

Phép so sánh (<, >, ==...) trên array cũng là ufunc, trả về array boolean cùng shape.

Boolean Array như một Mask

```
support_tickets = np.array([1, 4, 2, 5, 0, 3])
monthly_fee = np.array([12.0, 35.0, 18.0, 40.0, 9.0, 22.0])

high_risk_mask = (support_tickets >= 3) | (monthly_fee > 30)
print(high_risk_mask)                # [False  True False  True False False]

print(monthly_fee[high_risk_mask])    # chỉ lấy các giá trị thỏa điều kiện
print(monthly_fee[monthly_fee > 20]) # lọc trực tiếp
```

So sánh với `churn_risk()` viết bằng `if/else` ở Buổi 1: cùng logic điều kiện, nhưng boolean masking áp dụng cho *toàn bộ* tập dữ liệu cùng lúc, không cần vòng lặp.

Fancy Indexing

```
x = np.array([10, 20, 30, 40, 50])
indices = [0, 2, 4]
print(x[indices])           # array([10, 30, 50])

grid = np.arange(12).reshape(3, 4)
row = np.array([0, 1, 2])
col = np.array([2, 1, 3])
print(grid[row, col])       # lay tung cap (0,2),(1,1),(2,3)

print(grid[[0, 2]])         # lay hang 0 va hang 2
print(grid[grid > 6])       # ket hop voi boolean mask
```

Fancy indexing dùng một mảng chỉ số (hoặc boolean mask) để truy cập nhiều phần tử không liền kề trong một lệnh duy nhất – luôn trả về *copy*, khác với slicing.

Sorting Arrays

```
x = np.array([2, 1, 4, 3, 5])
print(np.sort(x))          # [1 2 3 4 5] - tra ve ban sao da sap xep
print(x.sort())            # sap xep tai cho (in-place)

x = np.array([2, 1, 4, 3, 5])
i = np.argsort(x)          # chi so sap xep tang dan
print(i)                   # [1 0 3 2 4]
print(x[i])                # ap dung chi so -> array da sap xep

grid = np.random.randint(0, 10, (4, 6))
print(np.sort(grid, axis=0)) # sap xep theo tung cot
```

Structured Arrays: cầu nối tới Pandas

```
data = np.zeros(3, dtype={
    "names": ("name", "age", "monthly_fee"),
    "formats": ("U10", "i4", "f8")
})
data["name"] = ["An", "Binh", "Chi"]
data["age"] = [25, 32, 41]
data["monthly_fee"] = [12.5, 35.0, 18.0]

print(data["name"])           # array(['An', 'Binh', 'Chi'])
print(data[data["age"] > 30]["name"]) # loc theo điều kiện
```

Kết nối tới Pandas

structured array cho phép mỗi "cột" mang tên và kiểu dữ liệu riêng – đây chính xác là ý tưởng nền tảng của `pandas.DataFrame`, sẽ được trình bày đầy đủ ở Buổi 3.

Tổng kết Module 7

- Boolean mask: `array[condition]` – lọc dữ liệu vectorized.
- Fancy indexing: truy cập nhiều phần tử qua mảng chỉ số, luôn trả về copy.
- `np.sort`, `np.argsort`: sắp xếp và lấy chỉ số sắp xếp.
- structured array: nền tảng khái niệm cho `pandas.DataFrame`.

Module 8: Ứng dụng NumPy trong Machine Learning

Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- 1 Cài đặt và huấn luyện mô hình hồi quy tuyến tính bằng Gradient Descent thuần NumPy.
- 2 Giải bài toán hồi quy tuyến tính bằng nghiệm dạng đóng (Normal Equation).
- 3 Giải thích phân rã SVD và dùng SVD để giải bài toán tối ưu bình phương tối thiểu (least squares) một cách ổn định.
- 4 So sánh ưu, nhược điểm của ba cách tiếp cận: Gradient Descent, Normal Equation, SVD/Pseudoinverse.

Hồi quy tuyến tính: một bài toán tối ưu

Cho tập dữ liệu (x_i, y_i) , tìm w, b sao cho $\hat{y}_i = wx_i + b$ xấp xỉ tốt nhất y_i .

Hàm mất mát (Loss Function)

$$J(w, b) = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2 = \frac{1}{n} \sum_{i=1}^n (wx_i + b - y_i)^2$$

Huấn luyện mô hình nghĩa là tìm (w, b) để *tối thiểu hóa* $J(w, b)$ – đây là ví dụ kinh điển nhất của bài toán tối ưu trong Machine Learning, liên hệ trực tiếp tới TrainingConfig và khái niệm epoch đã học ở Buổi 1.

Sinh dữ liệu tổng hợp bằng NumPy

```
import numpy as np

np.random.seed(42)
n = 200
X = np.random.rand(n, 1) * 10          # 1 feature, giá trị trong [0, 10)
true_w, true_b = 3.5, -2.0
noise = np.random.randn(n) * 1.0
y = true_w * X.squeeze() + true_b + noise

X_b = np.hstack([X, np.ones((n, 1))]) # thêm cột bias -> shape (n, 2)
```

Cột ones được ghép thêm vào X để hệ số b (bias/intercept) được xử lý cùng công thức ma trận với w – kỹ thuật chuẩn khi cài đặt hồi quy tuyến tính bằng đại số tuyến tính.

Huấn luyện bằng Gradient Descent

```
w = np.zeros(2)          # khoi tao tham so [w, b] = [0, 0]
lr = 0.01                 # learning rate
epochs = 1000

for epoch in range(epochs):
    y_pred = X_b @ w      # forward: du doan
    error = y_pred - y
    loss = np.mean(error ** 2) # MSE
    grad = (2 / n) * (X_b.T @ error) # dao ham cua J theo [w, b]
    w = w - lr * grad      # cap nhat tham so

print(w) # [3.4904, -1.8828] (gan voi true_w=3.5, true_b=-2.0)
```

Toàn bộ vòng lặp huấn luyện (epoch) được vector hóa bằng NumPy: $X_b @ w$ và $X_b.T @ error$ tính dự đoán và đạo hàm cho *toàn bộ* 200 mẫu cùng lúc, không cần vòng lặp qua từng điểm dữ liệu.

Nghiệm dạng đóng: Normal Equation

Công thức

$$\nabla J(w) = 0 \implies w^* = (X^T X)^{-1} X^T y$$

```
w_ne = np.linalg.inv(X_b.T @ X_b) @ X_b.T @ y  
print(w_ne)    # [3.4922, -1.8948] - gần giống kết quả Gradient Descent
```

Không cần vòng lặp huấn luyện: giải trực tiếp phương trình đạo hàm bằng 0. Tuy nhiên `np.linalg.inv` có thể **mất ổn định về số học** khi $X^T X$ gần suy biến (các feature tương quan tuyến tính mạnh – multicollinearity).

Singular Value Decomposition (SVD)

Định lý phân rã

Mọi ma trận $X \in \mathbb{R}^{n \times d}$ đều phân rã được thành:

$$X = U\Sigma V^T$$

- U ($n \times n$): các vector trực chuẩn (orthonormal) trong không gian output.
- Σ ($n \times d$): ma trận đường chéo chứa các *giá trị kỳ dị* (singular values) $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$.
- V ($d \times d$): các vector trực chuẩn trong không gian input.

SVD là công cụ nền tảng của đại số tuyến tính số (Golub & Van Loan, 2013), đứng sau least squares, PCA (Buổi 12) và nhiều thuật toán nén dữ liệu.

SVD trong NumPy

```
U, S, Vt = np.linalg.svd(X_b, full_matrices=False)
print(U.shape, S.shape, Vt.shape)    # (200, 2) (2,) (2, 2)
print(S)                             # cac gia tri ky di, giam dan
```

`np.linalg.svd` trả về U , mảng 1 chiều S (các giá trị kỳ dị, không phải ma trận đường chéo đầy đủ) và V^T (đã chuyển vị sẵn) – dùng `np.diag(S)` nếu cần dựng lại ma trận Σ đầy đủ.

Giải bài toán tối ưu bằng SVD: Pseudoinverse

Nghiệm bình phương tối thiểu qua SVD

$$w^* = X^+ y, \quad X^+ = V \Sigma^+ U^T$$

trong đó Σ^+ là nghịch đảo của từng giá trị kỳ dị khác 0 trên đường chéo.

```
S_inv = np.diag(1 / S)
w_svd = Vt.T @ S_inv @ U.T @ y
print(w_svd)                # [3.4922, -1.8948] - giống Normal Equation

w_pinv = np.linalg.pinv(X_b) @ y    # cách làm thực tế: dùng hàm có sẵn
print(w_pinv)
```

`np.linalg.pinv` tính pseudoinverse Moore-Penrose thông qua SVD, cho nghiệm *chuẩn tối thiểu* (minimum-norm) ổn định về số học ngay cả khi X không đủ hạng (rank-deficient).

Vì sao SVD ổn định hơn Normal Equation?

```
# them 1 cot gan nhu phu thuoc tuyen tinh voi cot dau (multicollinearity)
X_bad = np.hstack([X, X * 2 + 1e-10, np.ones((n, 1))])

print(np.linalg.cond(X_bad))          # ~6.6e15  (dieu kien cuc ky xau)

U2, S2, Vt2 = np.linalg.svd(X_bad, full_matrices=False)
print(S2)          # [1.795e+02  7.328e+00  2.70e-14] <- gia tri ky di gan 0!

w_bad_pinv = np.linalg.pinv(X_bad) @ y    # van chay on dinh, khong loi
```

Quan sát

SVD "lộ diện" trực tiếp nguyên nhân mất ổn định: một giá trị kỳ dị gần 0 nghĩa là X gần suy biến. $\text{np.linalg.inv}(X^T X)$ khuếch đại sai số này lên bình phương ($\text{cond}(X^T X) = \text{cond}(X)^2$), trong khi pinv xử lý ổn định nhờ ngưỡng hóa các giá trị kỳ dị rất nhỏ.

So sánh ba cách giải bài toán hồi quy tuyến tính

Phương pháp	Ưu điểm	Hạn chế
Gradient Descent	Mở rộng tốt cho dữ liệu lớn, deep learning	Cần chọn learning rate, nhiều epoch
Normal Equation	Nghiem đóng, không cần lặp	Không ổn định khi $X^T X$ gần suy biến
SVD / Pseudoinverse	Ổn định số học, xử lý được rank-deficient	Chi phí tính toán cao hơn với X rất lớn

Kết nối

`np.linalg.lstsq` (dùng nội bộ SVD) chính là cách `sklearn.linear_model.LinearRegression` giải bài toán bên dưới. SVD còn là nền tảng của PCA (Buổi 12) – sẽ gặp lại khái niệm giá trị kỳ dị khi giảm chiều dữ liệu.

Tổng kết Module 8

- Huấn luyện mô hình = giải bài toán tối ưu tối thiểu hóa hàm mất mát $J(w, b)$.
- Gradient Descent: cập nhật tham số lặp lại bằng đạo hàm, vector hóa hoàn toàn bằng NumPy.
- Normal Equation: nghiệm đóng $w^* = (X^T X)^{-1} X^T y$, nhanh nhưng có thể mất ổn định.
- SVD: $X = U \Sigma V^T$; pseudoinverse $X^+ = V \Sigma^+ U^T$ cho nghiệm ổn định, kể cả khi X gần suy biến.
- `np.linalg.inv`, `np.linalg.svd`, `np.linalg.pinv`, `np.linalg.lstsq` – bộ công cụ đại số tuyến tính số của NumPy.

Bảng tổng hợp các hàm NumPy quan trọng

Nhóm	Hàm/thao tác	Mục đích
Tạo array	<code>array</code> , <code>zeros</code> , <code>arange</code> , <code>random.*</code>	Khởi tạo dữ liệu
Thuộc tính	<code>shape</code> , <code>dtype</code> , <code>ndim</code> , <code>size</code>	Kiểm tra cấu trúc array
Indexing/Slicing	<code>x[i]</code> , <code>x[a:b]</code> , <code>x[:, j]</code>	Truy cập phần tử/subarray
Tính toán	<code>+</code> , <code>*</code> , <code>np.exp</code> , <code>np.sum</code> , <code>mean</code> , <code>std</code>	Ufunc & aggregation
Broadcasting	<code>(X - mean) / std</code>	Tính toán trên shape khác nhau
Lọc dữ liệu	<code>x[x > k]</code> , boolean mask	Lọc theo điều kiện
Sắp xếp	<code>np.sort</code> , <code>np.argsort</code>	Sắp xếp giá trị/chỉ số
Đại số tuyến tính	<code>np.linalg.inv</code> , <code>svd</code> , <code>pinv</code> , <code>lstsq</code>	Giải hệ phương trình, SVD, least squares

Bài tập thực hành gợi ý

- ❶ Dùng `%timeit` so sánh vòng lặp Python thuần và phép toán NumPy vectorized trên mảng 1 triệu phần tử.
- ❷ Tạo array ngẫu nhiên mô phỏng điểm số 100 sinh viên, tính trung bình, độ lệch chuẩn, và lọc ra danh sách sinh viên có điểm trên trung bình bằng boolean mask.
- ❸ Chuẩn hóa (standardize) một ma trận dữ liệu 2 chiều bằng broadcasting, không dùng vòng lặp `for`.
- ❹ Cài đặt hồi quy tuyến tính bằng Gradient Descent thuần NumPy trên dữ liệu tự sinh, sau đó đối chiếu kết quả với Normal Equation và `np.linalg.lstsq` (SVD).

Gợi ý mở rộng

Có thể thử áp dụng lên dữ liệu thực từ case study *US Baby Names* hoặc *Bitly/USA.gov* (Python for Data Analysis, Module 4) để làm quen với dữ liệu ngoài đời thực trước khi học Pandas.

- Harris, C. R., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362.
- Van der Walt, S., Colbert, S. C., & Varoquaux, G. (2011). The NumPy array: A structure for efficient numerical computation. *Computing in Science & Engineering*, 13(2), 22–30.
- Pérez, F., & Granger, B. E. (2007). IPython: A system for interactive scientific computing. *Computing in Science & Engineering*, 9(3), 21–29.
- Golub, G. H., & Van Loan, C. F. (2013). *Matrix Computations* (4th ed.). Johns Hopkins University Press.
- VanderPlas, J. (2016). *Python Data Science Handbook: Essential Tools for Working with Data*. O'Reilly Media.
- McKinney, W. (2022). *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter* (3rd ed.). O'Reilly Media.
- Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

Chuẩn bị cho buổi tiếp theo: Pandas Foundation

Thông điệp

`pandas.Series` và `pandas.DataFrame` được xây dựng trên nền `ndarray`: mọi kỹ thuật vectorization, broadcasting và boolean masking vừa học sẽ tiếp tục áp dụng trực tiếp trong Pandas.